

Introduction to Programming

CSC1103 &1104

Lecture-10

American International University Bangladesh
Dept. of Computer Science
Faculty of Science and Information Technology

Outline

- Standard Input and Output
  - Standard Input and Output (I/O)– Review & more
    - Buffered/unbuffered input
    - Character I/O
    - Formatted I/O
  - Redirecting I/O to files. Pipes

Standard Input and Output

- input and output devices: such as keyboards, disk drives, printers, screen.
- ***The C language itself does not have any special statements for performing input/output (I/O) operations; all I/O operations in C must be carried out through function calls*.**
- **These functions are contained in the standard C library:**
- **`#include <stdio.h>`**
- ***Advantages***:
  - ***Portability***: they work in a wide variety of computer environments
  - ***General character***: they generalize to using *files* for I/O
- Disadvantage:
  - Less performance: they don't take advantage of features peculiar to a particular system.

Kinds of Standard I/O functions

- Character I/O
  - getchar, putchar
- Formatted I/O
  - scanf, printf

Character I/O example

```
/* echo.c -- repeats input */
#include <stdio.h>
int main(void) {
  char ch;
  while ((ch = getchar() ) != '*')
    putchar(ch);
  return 0;
}
```

I/O streams

- C treats input and output devices the same as it treats regular files on storage devices. In particular, the keyboard and the display devices are treated as files opened automatically by every C program.
- Conceptually, the C program deals with a *stream* instead of directly with a file. A ***stream*** is an idealized flow of data to which the actual input or output is mapped.
- Keyboard input is represented by a stream called `stdin`, and output to the screen is represented by a stream called `stdout`. The `getchar()`, `putchar()`, `printf()`, and `scanf()` functions are all members of the standard I/O package, and they deal with these two streams.

EOF

- One implication of I/O streams is that you can use the same techniques with keyboard input as you do with files.
- For example, a program reading a file needs a way to detect the end of the file so that it knows where to stop reading. Therefore, C input functions come with a built-in, end-of-file detector. Because keyboard input is treated like a file, you should be able to use that end-of-file detector to terminate keyboard input, too.
  - CTRL-Z is EOF for keyboard input

Character I/O example + EOF

```
/* echo_eof.c -- repeats input */
#include <stdio.h>
int main(void) {
  char ch;
  while ((ch = getchar() ) !=  EOF)
    putchar(ch);
  return 0;
}
```

Buffered/unbuffered input

```
/* echo.c -- repeats input */
#include <stdio.h>
int main(void) {
  char ch;
  while ((ch = getchar()) != '*')
    putchar(ch);
  return 0;
}
```

Suppose you type: `Hi!*`

What exactly does the program run look like ?

Buffered/unbuffered input

```
/* echo.c -- repeats input */
#include <stdio.h>
int main(void) {
  char ch;
  while ((ch = getchar()) != '*')
    putchar(ch);
  return 0;
}
```

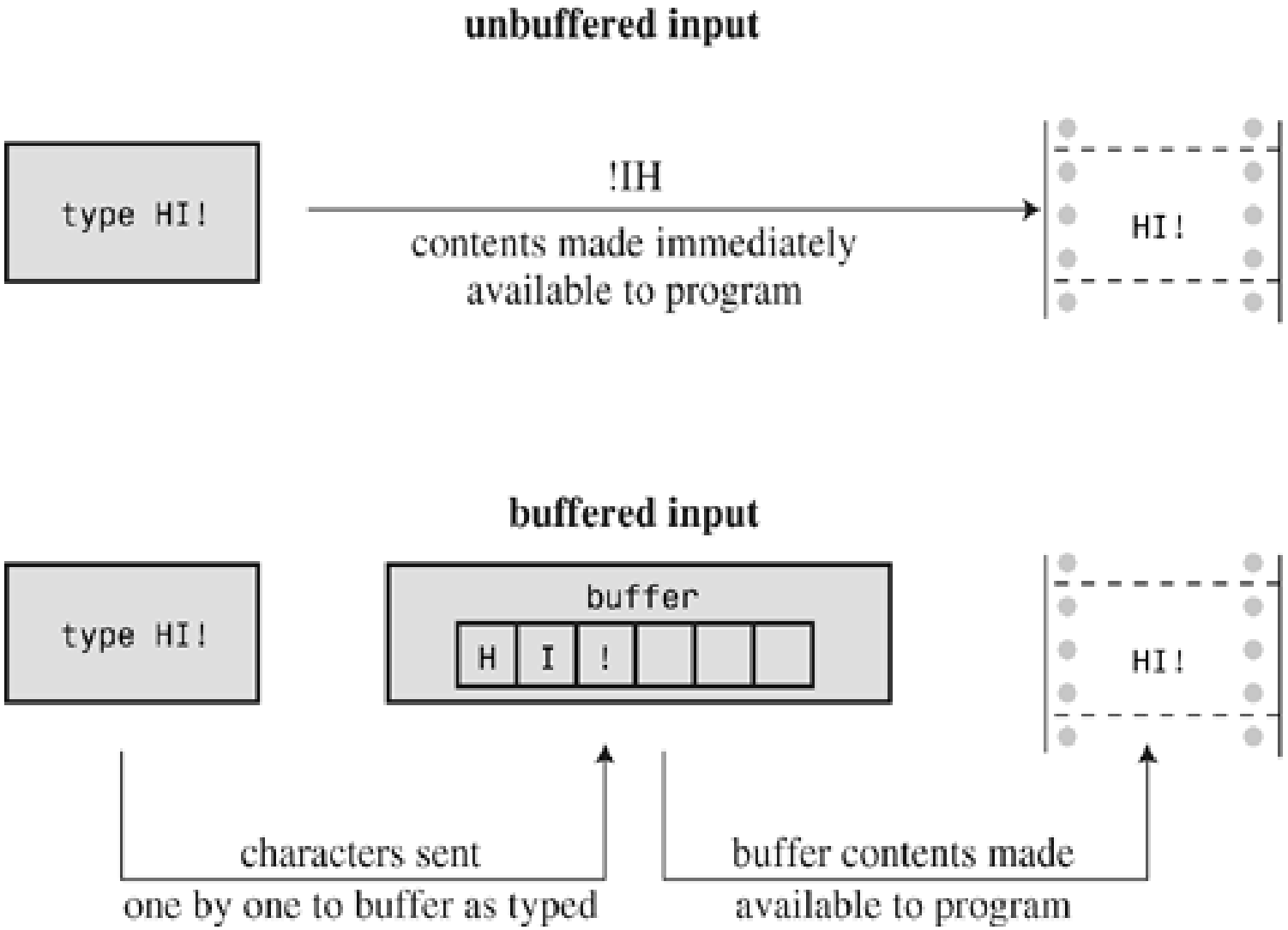
Suppose you type: `Hi!*`
What does the program run look like ?

`HHii!!*` ← OR → `Hi!*` `Hi!`

If the system is Unbuffered

If the system is Buffered

most systems are *line-buffered*: input buffer is emptied only after pressing ENTER



From [C Primer Plus]

Buffered/unbuffered input in C

- ANSI C: `<stdio.h>` functions (`getchar()`) should be buffered
- Additional libraries may provide unbuffered input
  - `<conio.h>` offers `getche()` for echoed unbuffered input and `getch()` for unechoed unbuffered input
- The way of buffereing may be controled by the operating system
  - Unix: the `ioctl()` function (part of the Unix library but not part of standard C) can specify the type of input you want, and `getchar()` behaves accordingly

Formatted Output - `printf`

The output function `printf` *translates internal values to characters and prints them to the standard output*.

A formal declaration of the `printf` function:

```
int printf(const char *format,...);
```

printf converts, formats, and prints its arguments on the standard output under control of the format. It *returns the number of characters printed*.

printf is a ***function with variable number of arguments (this is possible in C !)***: the declaration with 3 points (…) means that the number and types of these arguments may vary. The declaration ... can only appear at the end of an argument list.

printf

The general format of a `printf` conversion specification is as follows:

`%[flags][width][.prec][hlL]type`

`type` represents the *conversion characters*, it is a mandatory field.

Optional fields (enclosed in brackets) are `flags`, `width` and `prec`, and the type modifiers `[hlL]`; if they are used, they must appear in the order shown.

References [Kochan] (to look up only !)

Table 16.1 `printf` **Flags**

Table 16.2 `printf` **Width and Precision Modifiers**

Table 16.3 `printf` **Type Modifiers**

Table 16.4 `printf` **Conversion Characters**

Char	Use to Display
`i` or `d`	Integer
`u`	Unsigned integer
`o`	Octal integer
`x`	Hexadecimal integer, using a–f
`X`	Hexadecimal integer, using A–F
`f` or `F`	Floating-point number, to six decimal places by default
`e` or `E`	Floating-point number in exponential format (`e` places lowercase e before the exponent, `E` places uppercase E before exponent)
`g`	Floating-point number in `f` or `e` format
`G`	Floating-point number in `F` or `E` format
`a` or `A`	Floating-point number in the hexadecimal format 0x*d.dddd*p±*d*
`c`	Single character
`s`	Null-terminated character string
`p`	Pointer
`n`	Doesn't print anything; stores the number of characters written so far by this call inside the `int` pointed to by the corresponding argument (see note from Table 16.3)
`%`	Percent sign

Table 16.1 `printf` **Flags**

Flag	Meaning
`-`	Left-justify value
`+`	Precede value with `+` or `-`
`(space)`	Precede positive value with space character
`0`	Zero fill numbers
`#`	Precede octal value with `0`, hexadecimal value with `0x` (or `0X`); display decimal point for floats; leave trailing zeroes for `g` or `G` format

Table 16.2 `printf` **Width and Precision Modifiers**

Specifier	Meaning
number	Minimum size of field
*	Take next argument to `printf` as size of field
.*number*	Minimum number of digits to display for integers; number of decimal places for `e` or `f` formats; maximum number of significant digits to display for `g`; maximum number of characters for `s` format
.*	Take next argument to `printf` as precision (and interpret as indicated in preceding row)

Table 16.3 `printf` **Type Modifiers**

Type	Meaning
`hh`	Display integer argument as a character
`h`*	Display `short integer`
`l`*	Display `long integer`
`ll`*	Display `long long integer`
`L`	Display `long double`
`j`*	Display `intmax_t` or `uintmax_t` value
`t`*	Display `ptrdiff_t` value
`z`*	Display `size_t` value

Formatted Input - `scanf`

The function `scanf` is the input analog of printf, providing many of the same conversion facilities in the opposite direction.

```
int scanf(const char *format, ...);
```

`scanf` **reads** characters from the standard input, i**nterprets** them according to the specification in format, and **stores** the results through the remaining arguments (*each of which must be a pointer)*.

`scanf` stops when it exhausts its format string, or when some input fails to match the control specification.

It returns as its value the number of successfully matched and assigned input items. This can be used to decide how many items were found. On the end of file, EOF is returned; note that this is different from 0, which means that the next input character does not match the first specification in the format string.

The next call to scanf resumes searching immediately after the last character already converted.

scanf

As with `printf`, `scanf` takes optional modifiers between the % and the conversion character.

References [Kochan] (to look up only !)

Table 16.5 `scanf` **Conversion Modifiers**

Table 16.6 `scanf` **Conversion Characters**

Some interesting scanf conversions

`s`	The value to be read is a sequence of characters; the sequence begins with the first nonwhitespace character and is terminated by the first whitespace character. The corresponding argument is a pointer to a character array, which must contain enough characters to contain the characters that are read plus the null character that is automatically added to the end. If a number precedes the `s`, the specified number of characters is read, unless a whitespace character is encountered first.
`[...]`	Characters enclosed within brackets indicate that a character string is to be read, as in `%s`; the characters within the brackets indicate the permissible characters in the string. If any character other than that specified in the brackets is encountered, the string is terminated; the sense of how these characters are treated can be "inverted" by placing a `^` as the first character inside the brackets. In such a case, the subsequent characters are taken to be the ones that will terminate the string; that is, if any of the subsequent characters are found on the input, the string is terminated.
`n`	Nothing gets read. The number of characters read so far by this call is written into the `int` pointed to by the corresponding argument.

scanf Examples (1)

Whitespace characters inside a format string match an arbitrary number of whitespace characters on the input. So, the call

```
scanf ("%i%c", &i, &c);
```

with the line of text

```
29 w
```

assigns the value `29` to `i` and a space character to `c` because this is the character that appears immediately after the characters `29` on the input.

If the following `scanf` call is made instead:

```
scanf ("%i %c", &i, &c);
```

and the same line of text is entered, the value `29` is assigned to `i` and the character `'w'` to `c` because the blank space in the format string causes the `scanf` function to ignore any leading whitespace characters after the characters `29` have been read.

scanf Examples (2)

An asterisk can be used to skip fields. If the `scanf` call

```
scanf ("%i %5c %*f %s", &i1, text, string);
```

is executed and the following line of text is typed in:

```
144abcde 736.55 (wine and cheese)
```

the value `144` is stored in `i1`; the five characters `abcde` are stored in the character array `text`; the floating value `736.55` is matched but not assigned; and the character string `"(wine"` is stored in `string`, terminated by a `null`.

The next call to `scanf` *picks up where the last one left off*. So, a subsequent call such as `scanf ("%s %s %i", string2, string3, &i2);` has the effect of storing the character string `"and"` in `string2` and the string `"cheese)"` in `string3` and further waits for an integer to be typed in.

scanf Examples (3)

The `scanf` call

```
scanf ("%[^/]", text);
```

indicates that **the string to be read can consist of any character except for** a slash. Using the preceding call on the following line of text

```
(wine and cheese)/
```

has the effect of storing the string "`(wine and cheese)`" in `text` because the string is not terminated until the `/` is matched (which is also the character read by `scanf` on the next call).

To read an entire line from the terminal into the character array `buf`, you can specify that the newline character at the end of the line is your string terminator:

```
scanf ("%[^\n]\n", buf);
```

The newline character is repeated outside the brackets so that `scanf` matches it and does not read it the next time it's called. (Remember, `scanf` always continues reading from the character that terminated its last call.)

scanf Examples (4)

When a value is read that does not match a value expected by `scanf` *(for example, typing in the character* `x` *when an integer is expected),* `scanf` *does not read any further items from the input and immediately returns*. Because the **function returns the number of items that were successfully read and assigned** to variables in your program, this value can be tested to determine if any errors occurred on the input. For example, the call

```
if ( scanf ("%i %f %i", &i, &f, &l) != 3 )
   printf ("Error on input\n");
```

tests to make certain that `scanf` successfully read and assigned three values. If not, an appropriate message is displayed.

Remember, the return value from `scanf` indicates the number of values read *and assigned*, so the call

```
scanf ("%i %*d %i", &i1, &i3)
```

returns 2 when successful and not 3 because you are reading and assigning *two* integers (skipping one in between). Note also that the use of `%n` (to obtain the number of characters read so far) does not get included in the value returned by `scanf`.

Redirecting I/O to a file

- Sometimes we want programs to take input from a file instead from the keyboard or to write results in a file instead on the screen
- Both read and write file operations can be easily performed under many operating systems, such as Unix and Windows, without anything special being done at all to the program through **I/O redirecting**
- stdio functions in C have a general character, they work on abstract **streams**
- There are also special **file access** mechanisms are provided in C, but these will be discussed next semester

Redirect output

- For example, if you want to write all your program results into a file called `data.txt`:
  - all that you need to do under Unix or Windows, if running in a terminal window, is to *redirect the output* from the program `prog` into the file `data.txt` by executing the program with the following command at the command prompt:

```
prog > data.txt
```

- This command instructs the system to execute the program `prog` but to redirect the output normally written to the terminal into a file called `data.txt` instead.
- Any values displayed by `putchar` or `printf` do not appear on screen but are instead written into the file called `data.txt`.

Redirect input

- If you want your program to read all input from a file instead of the keyboard:
- You can have the program get its input from a file called `input.txt`, for example, by *redirecting the input* when the program is executed. If the program is called `prog`, the following command line works:

```
prog < input.txt
```

- Any call to a function that normally reads data from your window, such as `scanf` and `getchar`, will be made to read its information from the file `input.txt`

Redirect both input and output

- Redirect both input and output

```
prog < input.txt > data.txt
```

I/O redirection example

```
/* pecho.c -- repeats input */

#include <stdio.h>
int main(void) {
  char ch;
  while ((ch = getchar()) != '*')
    putchar(ch);
  return 0;
}
```

F1.txt

```
Bla bla bla
Oh la la *
Hoo hoo hoo
```

What is the result of running following command ?

```
pecho  <f1.txt >f2.txt
```

I/O redirection example 2

```
/* pecho.c -- repeats input */

#include <stdio.h>
int main(void) {
  char ch;
  while ((ch = getchar()) != '*')
    putchar(ch);
  return 0;
}
```

F3.txt

```
Bla bla bla
Oh la la
Hoo hoo hoo
```

What is the output of running following command ?

```
pecho  <f3.txt >f4.txt
```

Pipes

- Pipes: putting standard output of `prog` directly into the standard input of `anotherprog`

```
prog | anotherprog
```

Pipes example

prog1.c

```
#include <stdio.h>

int main(void) {
    char c;
    for (c='a'; c<='z'; c++)
        putchar(c);
}
```

prog2.c

```
#include <stdio.h>

int main(void) {
char c;
while ((c=getchar())!=EOF)
          printf("*%c",c);
}
```

What is the output of each command run ?

1. `prog1`
2. `prog2`
3. `prog1 | prog2`
4. `prog1 > temp.txt`
5. `prog2 < temp.txt`

Pipes example 2

pprintf.c

```
#include <stdio.h>

int main(void) {
    int i;
    for (i=1000; i<=10000;
i+=1000)
        printf("%d ",i);
}
```

pscanf.c

```
#include <stdio.h>

int main(void) {
  int i;
  while (scanf("%d ",&i)==1)
     printf("read %d \n",i);
}
```

Pipe:

```
pprintf | pscanf
```

int to string

string to int

Pipes example 3

pprintf.c

```
#include <stdio.h>

int main(void) {
    int i;
    for (i=1000; i<=10000;
i+=1000)
        printf("%d ",i);
}
```

prog2.c

```
#include <stdio.h>

int main(void) {
char c;
while ((c=getchar())!=EOF)
          printf("*%c",c);
}
```

Pipe:

```
pprintf | prog2
```

Comments on I/O redirection

- Note that I/O redirection is not actually part of the ANSI definition of C. This means that you might find operating systems that don't support it. Luckily, most do.
- **Special Functions for Working with Files:** situations do arise when you need more flexibility to work with files.
  - For example:
    - you might need to ***read data from two or more different files or to write output results into several different files***.
    - you might need to write numerical data into a more efficient ***binary file format***, not as text files
  - To handle these situations, special functions have been designed expressly for working with files. These will be discussed in another chapter